Dynamic File Compression Utility - Project Documentation

Project Title
Dynamic File Compression Utility
Objective
Build a smart, cross-platform compressor that auto-selects the best lossless codec (gzip/bz2/lzma/zstd/brotli) per file type and content, supports streaming & chunked parallel compression, optional dictionary training, and exposes a CLI + Python API with clear speed/ratio trade-offs.
Prompt
You are an expert Data Structures & Algorithms Engineer, Python/C++ Developer, File Compression Specialist, System Design Mentor, and GitHub Mentor.

Your task is to help me build a complete industry-oriented DSA project titled:

“Dynamic File Compression Utility”

IMPORTANT CONTEXT:
- I am a student.
- I want this project to act as proof of work on GitHub.
- This is a DSA course project.
- I want full code, explanation, virtual simulation, GitHub steps, README, screenshots, and interview preparation.
- I want this project to be useful for Software Developer, DSA, Backend Developer, System Programming, and Coding Interview roles.

NOW DO EVERYTHING STEP BY STEP.

1■■ PROJECT EXPLANATION

Explain clearly:

- What is a Dynamic File Compression Utility?
- What problem does it solve?
- Why file compression is important?
- How compression reduces file size
- How Huffman Coding is used for compression
- How this project demonstrates DSA concepts

Explain:
- simple explanation
- technical explanation

Workflow:
input file → frequency calculation → Huffman Tree creation → code generation → file compression → compressed output → decompression → original file recovery

---
2■■ TECH STACK OPTIONS

Give 3 options:

Option A: Easy
Option B: Intermediate
Option C: Advanced

Prefer:
- Python or C++
- Huffman Coding
- Priority Queue / Min Heap
- Binary Tree
- Dictionary/HashMap
- File Handling
- CLI interface

Then select the best option for students.

3■■ PROJECT ARCHITECTURE

Design architecture:

Input:
- text file or sample string

Processing:
- read file
- calculate character frequency
- build min heap
- build Huffman tree
- generate binary codes
- compress file
- decompress file

Output:
- compressed file
- decompressed file
- compression ratio report

Include:
- text-based architecture diagram
- algorithm flow
- data structure explanation

4■■ IMPLEMENTATION PLAN

Create phase-wise plan:

Phase 1: Setup
Phase 2: File reading
Phase 3: Frequency table creation
Phase 4: Min Heap implementation
Phase 5: Huffman Tree creation
Phase 6: Huffman code generation
Phase 7: File compression
Phase 8: File decompression
Phase 9: Compression ratio calculation
Phase 10: GitHub upload

For each phase explain:
- what to do
- why
- expected output
- common beginner mistakes

5■■ FOLDER STRUCTURE

Create GitHub-ready structure:

Dynamic-File-Compression-Utility/
■
■■■ input_files/
■■■ compressed_files/
■■■ decompressed_files/
■■■ src/
■■■ outputs/
■■■ images/
■■■ docs/
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ main.py

Explain each folder.

6■■ INSTALLATION GUIDE

Explain:
- Python/C++ setup
- required libraries
- how to run from terminal
- commands for Windows/Mac/Linux

7■■ FULL PROJECT CODE

Provide complete working code:

Include:
- reading input file
- character frequency calculation
- Node class for Huffman Tree
- min heap / priority queue logic
- Huffman tree construction
- binary code generation
- file compression
- file decompression
- compression ratio calculation
- command-line interface

Code must be:
- beginner-friendly
- modular

- commented
- executable

8■■ VIRTUAL SIMULATION

Explain how to simulate file compression:

- create sample text file
- calculate character frequencies
- show Huffman codes
- compress file
- compare original and compressed size
- decompress file
- verify original and decompressed file match

Mention:
- screenshots to capture
- outputs to save
- reports to upload on GitHub

9■■ HOW TO RUN PROJECT

Explain:
- commands to run
- sample input file
- expected terminal output
- compressed file output
- decompressed file output
- compression ratio output

■ GITHUB UPLOAD STEPS

Explain:
- create repository
- push code
- commit messages
- upload screenshots and output files

Give:
- best repo name
- description
- GitHub tags

1■■1■■ README.md

Generate full README:

- project overview
- problem statement
- DSA concepts used
- algorithm explanation
- features

- folder structure
- how to run
- sample output
- screenshots
- learning outcomes

1■■2■■ PROOF BUILDING STRATEGY

Create day-wise proof plan:

Day 1: setup and file reading
Day 2: frequency table
Day 3: heap and Huffman tree
Day 4: encoding/compression
Day 5: decoding/decompression
Day 6: compression ratio and GitHub documentation

Include:
- what to commit
- commit messages
- screenshots to save

1■■3■■ SCREENSHOTS / OUTPUTS

Tell exactly what to capture:

- project folder screenshot
- input file screenshot
- frequency table output
- Huffman codes output
- compression terminal output
- compressed file generated
- decompressed file generated
- compression ratio output
- GitHub repo preview

1■■4■■ INTERVIEW PREPARATION

Give:
- 10 interview questions
- strong answers
- HR explanation
- technical explanation

IMPORTANT:
One question must be:
“Explain your project.”

FINAL RULES:
- Do not skip code
- Make it beginner-friendly
- Make it executable
- Make it GitHub-ready

- Provide copy-paste ready content
Industry Relevance
Backups, CI/CD artifacts, data lakes, and log shipping all juggle size vs. speed. A content-aware utility that tunes codecs and levels in real time cuts storage and egress costs while keeping throughput high.

Step-by-Step Implementation
Step 1 — Scope, Modes & Architecture
What are we doing?
Designing a modular tool: Detector → Strategy (codec/level) → Compressor (streaming, parallel) → Verifier → Metadata (.dfc manifest) with CLI/API.
Why are we doing it?
Different files compress differently (CSV vs PNG vs Parquet). Auto-selection improves both ratio and speed without manual tuning.
How do we execute this?
Codecs: gzip, bz2, lzma/xz, zstd, brotli (lossless).

Modes: auto, fast, max, balanced, archive (tar+compress), ndjson (frame-aware).

Artifacts: .dfc manifest storing original size, checksum, codec, level, dictionary id.

Stack: Python 3.10+, zstandard, brotli, stdlib gzip, bz2, lzma, xxhash or hashlib. CLI via argparse, API via module.

ChatGPT regeneration prompt
“Sketch a compression pipeline with detector → strategy → compressor → verifier and a .dfc manifest.”

Step 2 — Content Detection & Sampling
What are we doing?
Identifying file type and basic entropy from a small sample to pick a codec and level.
Why are we doing it?
Texty/logs favor zstd/brotli; already-compressed (jpeg/png/mp4) should be stored as-is or repacked losslessly.
How do we execute this?
Detect by magic bytes & extension fallback.

Compute sample entropy, line endings, column separators.

Heuristics table to pick codec.

```
# detector.py
import mimetypes, struct, zlib, pathlib
MAGIC = {
b"\x1F\x8B": "gzip", b"\x42\x5A\x68": "bz2", b"\xFD\x37\x7A\x58\x5A\x00": "xz",
b"\x28\xB5\x2F\xFD": "zstd", b"\xFF\xD8\xFF": "jpeg", b"\x89PNG\r\n\x1a\n": "png"
}
def magic_type(path: str) -> str|None:
with open(path,'rb') as f:
```

```
head=f.read(16)
for m,t in MAGIC.items():
if head.startswith(m): return t
return None

def sample_stats(path: str, n=256*1024):
import math
with open(path,'rb') as f: buf = f.read(n)
if not buf: return {"entropy":0,"text_ratio":0,"nl":0}
# entropy
from collections import Counter
c=Counter(buf); total=len(buf)
H=-sum((v/total)*math.log2(v/total) for v in c.values())
# crude text test
text_ratio = sum(32<=b<=126 or b in (9,10,13) for b in buf)/total
nl = buf.count(b'\n')
return {"entropy":H,"text_ratio":text_ratio,"nl":nl}

def guess_mime(path: str) -> str:
mt, _ = mimetypes.guess_type(path)
return mt or "application/octet-stream"
```

ChatGPT regeneration prompt
“Write a detector that returns magic type, MIME guess, and sample entropy/text ratio for a file.”

Step 3 — Strategy Selection (Codec & Level)
What are we doing?
Mapping detection signals to {codec, level, chunk_size, threads}.
Why are we doing it?
Encode the speed/ratio trade-off in one place; easy to tune.
How do we execute this?
Text/logs (high text_ratio, many newlines): zstd level 6–9 or brotli 5–9.

CSV/JSON/NDJSON: zstd 10 (dict optional).

Already-compressed media: store (maybe tar + zstd at low level for many small files).

Binaries with low entropy: lzma 6–7 for max ratio (offline).

fast mode favors zstd 3–5 or gzip 3; max favors lzma 7 or zstd 19 (careful CPU).

```
# strategy.py
from dataclasses import dataclass
@dataclass
class Plan: codec:str; level:int; chunk:int; threads:int; store:bool=False; dict_id:str|None=None

def choose_strategy(path:str, mode="auto"):
from detector import magic_type, sample_stats, guess_mime
mt = magic_type(path)
if mt in {"gzip","bz2","xz","zstd"}: return Plan(codec="store", level=0, chunk=0, threads=0,
store=True)
```

```
mime = guess_mime(path); s = sample_stats(path)
texty = s["text_ratio"]>0.7 and s["entropy"]<7.7
if mode=="fast": return Plan(codec="zstd", level=3, chunk=1024*1024, threads=0)
if mode=="max": return Plan(codec="lzma", level=7, chunk=1024*1024, threads=0)
if "text" in mime or texty or mime in {"application/json","text/csv"}:
return Plan(codec="zstd", level=8, chunk=4*1024*1024, threads=0)
if mime.startswith("image/") or mime.startswith("video/"):
return Plan(codec="store", level=0, chunk=0, threads=0, store=True)
# default balanced
return Plan(codec="zstd", level=6, chunk=2*1024*1024, threads=0)
```

ChatGPT regeneration prompt
“Create a strategy chooser that returns codec/level/chunk/threads given MIME + entropy + mode.”

Step 4 — Streaming Compressors (Chunked, Parallel)
What are we doing?
Implementing memory-safe, large-file streaming with optional parallelism.
Why are we doing it?
Avoids loading whole files; boosts throughput on multi-core.
How do we execute this?
Use stdlib gzip/bz2/lzma streams; third-party zstandard.ZstdCompressor, brotli for streaming.

Parallel: split by chunk boundaries, compress in workers, write concatenated zstd skippable frames (or tar→single stream).

```
# compress.py
from concurrent.futures import ThreadPoolExecutor
import gzip, bz2, lzma, brotli, zstandard as zstd, pathlib, os, hashlib, json, time
from strategy import choose_strategy, Plan

def sha256_file(path):
h=hashlib.sha256()
with open(path,'rb') as f:
for b in iter(lambda: f.read(1<<20), b''):
h.update(b)
return h.hexdigest()

def compress_file(src: str, dst: str|None=None, mode="auto") -> dict:
p=pathlib.Path(src); plan = choose_strategy(src, mode)
if plan.store:
dst = dst or (str(p)+".store")
os.link(src, dst) if hasattr(os, "link") else open(dst,"wb").write(open(src,"rb").read())
return _manifest(src, dst, plan, 0)

if plan.codec=="zstd":
cctx = zstd.ZstdCompressor(level=plan.level, threads=plan.threads or 0)
dst = dst or (str(p)+".zst")
with open(src,'rb') as fi, open(dst,'wb') as fo:
with cctx.stream_writer(fo) as zw:
for chunk in iter(lambda: fi.read(plan.chunk), b''):
zw.write(chunk)
return _manifest(src, dst, plan, 0)
```

```
if plan.codec=="brotli":
dst = dst or (str(p)+".br")
enc = brotli.Compressor(quality=plan.level)
with open(src,'rb') as fi, open(dst,'wb') as fo:
for chunk in iter(lambda: fi.read(plan.chunk), b''):
fo.write(enc.process(chunk))
fo.write(enc.finish())
return _manifest(src, dst, plan, 0)

if plan.codec=="gzip":
dst = dst or (str(p)+".gz")
with open(src,'rb') as fi, gzip.open(dst,'wb', compresslevel=plan.level) as fo:
for chunk in iter(lambda: fi.read(plan.chunk), b''): fo.write(chunk)
return _manifest(src, dst, plan, 0)

if plan.codec=="bz2":
dst = dst or (str(p)+".bz2")
with open(src,'rb') as fi, bz2.open(dst,'wb', compresslevel=min(9,plan.level)) as fo:
for chunk in iter(lambda: fi.read(plan.chunk), b''): fo.write(chunk)
return _manifest(src, dst, plan, 0)

if plan.codec=="lzma":
dst = dst or (str(p)+".xz")
with open(src,'rb') as fi, lzma.open(dst,'wb', preset=min(9,plan.level)) as fo:
for chunk in iter(lambda: fi.read(plan.chunk), b''): fo.write(chunk)
return _manifest(src, dst, plan, 0)

def _manifest(src, dst, plan: Plan, dict_size):
man = {
"source": src, "output": dst, "codec": plan.codec, "level": plan.level,
"chunk": plan.chunk, "dict_id": plan.dict_id, "sha256": sha256_file(src),
"orig_bytes": pathlib.Path(src).stat().st_size, "out_bytes": pathlib.Path(dst).stat().st_size
}
with open(str(dst)+".dfc.json","w") as f: json.dump(man,f,indent=2)
return man
```

ChatGPT regeneration prompt
“Implement streaming compressors for zstd/brotli/gzip/bz2/lzma with manifest writing.”

Step 5 — Optional Dictionary Training (Texty Corpora)
What are we doing?
Training small zstd dictionaries from sample corpora to boost ratios on repetitive domain text (e.g., logs, JSON).
Why are we doing it?
Dictionaries improve compression of many small similar files and first-kilobyte patterns.
How do we execute this?

```
# dict_train.py
import zstandard as zstd, pathlib, random
def train_dict(glob_pattern="samples/*.log", dict_size=112*1024):
paths = list(pathlib.Path(".").glob(glob_pattern))
samples=[]
for p in paths:
b=p.read_bytes()
for _ in range(min(50, len(b)//4096)):
i=random.randrange(0, max(1,len(b)-4096))
samples.append(b[i:i+4096])
d=zstd.train_dictionary(dict_size, samples)
```

```
did=f"zstd_{dict_size}_{len(paths)}"
pathlib.Path(f"{did}.dict").write_bytes(d.as_bytes())
return did
```

ChatGPT regeneration prompt
“Create a zstd dictionary trainer that samples many small chunks and writes a .dict file.”

Step 6 — Verify & Decompress
What are we doing?
Ensuring round-trip integrity and exposing a uniform decompressor.
Why are we doing it?
Safety and confidence for pipelines and backups.
How do we execute this?

```
# verify.py
import gzip, bz2, lzma, brotli, zstandard as zstd, json, hashlib
from pathlib import Path

def sha256_stream(fp):
h=hashlib.sha256()
for b in iter(lambda: fp.read(1<<20), b""): h.update(b)
return h.hexdigest()

def decompress(src, dst=None):
if src.endswith(".zst"):
d=zstd.ZstdDecompressor()
with open(src,'rb') as fi, open(dst or src[:-4],'wb') as fo:
with d.stream_reader(fi) as dr:
for b in iter(lambda: dr.read(1<<20), b''): fo.write(b)
elif src.endswith(".br"):
with open(src,'rb') as fi, open(dst or src[:-3],'wb') as fo:
import brotli; fo.write(brotli.decompress(fi.read()))
elif src.endswith(".gz"):
with gzip.open(src,'rb') as fi, open(dst or src[:-3],'wb') as fo: fo.write(fi.read())
elif src.endswith(".bz2"):
with bz2.open(src,'rb') as fi, open(dst or src[:-4],'wb') as fo: fo.write(fi.read())
elif src.endswith(".xz"):
with lzma.open(src,'rb') as fi, open(dst or src[:-3],'wb') as fo: fo.write(fi.read())
else:
raise ValueError("Unknown extension")

def verify(manifest_path: str) -> bool:
man=json.loads(Path(manifest_path).read_text())
out = man["output"]; src = man["source"]
# round-trip check
tmp = out + ".verify.tmp"
decompress(out, tmp)
ok = sha256_stream(open(tmp,'rb')) == man["sha256"]
Path(tmp).unlink(missing_ok=True)
return ok
```

ChatGPT regeneration prompt
“Write a decompressor + manifest verifier that round-trips and checks SHA-256.”

Step 7 — Command-Line Interface
What are we doing?
Providing an ergonomic CLI for single files and folders.

Why are we doing it?
Students, CI, and scripts should be able to use it instantly.
How do we execute this?

```
# cli.py
import argparse, pathlib, sys, time
from compress import compress_file
from verify import verify, decompress
from dict_train import train_dict

def main():
ap=argparse.ArgumentParser(prog="dfc")
sub=ap.add_subparsers(dest="cmd")
c=sub.add_parser("compress"); c.add_argument("path"); c.add_argument("--mode",
choices=["auto","fast","balanced","max"], default="auto")
d=sub.add_parser("decompress"); d.add_argument("path")
v=sub.add_parser("verify"); v.add_argument("manifest")
t=sub.add_parser("train-dict"); t.add_argument("--glob", default="samples/*.log");
t.add_argument("--size", type=int, default=112*1024)
args=ap.parse_args()

if args.cmd=="compress":
t0=time.time()
man=compress_file(args.path, mode=args.mode)
print(f"OK {man['output']} ratio={man['out_bytes']/man['orig_bytes']:.3f} codec={man['codec']}
level={man['level']} {time.time()-t0:.2f}s")
elif args.cmd=="decompress":
decompress(args.path)
print("OK decompressed")
elif args.cmd=="verify":
print("OK" if verify(args.manifest) else "FAIL")
elif args.cmd=="train-dict":
did=train_dict(args.glob, args.size); print(f"dict saved: {did}.dict")
else:
ap.print_help()

if __name__=="__main__":
main()
```

ChatGPT regeneration prompt
“Build a dfc CLI with subcommands: compress, decompress, verify, train-dict.”

Step 8 — Batch/Archive Mode (Many Small Files)
What are we doing?
Packing folders into a tar stream then compressing (best for tiny files).
Why are we doing it?
Many small files compress and stream better when batched.
How do we execute this?

```
# archive.py
import tarfile, io, zstandard as zstd, pathlib
def compress_folder(folder: str, dst: str|None=None, level=8):
dst = dst or (pathlib.Path(folder).name + ".tar.zst")
cctx = zstd.ZstdCompressor(level=level)
with open(dst,'wb') as fo, cctx.stream_writer(fo) as zw:
with tarfile.open(mode="w|", fileobj=zw) as tar:
tar.add(folder, arcname=pathlib.Path(folder).name)
return dst
```

ChatGPT regeneration prompt
“Implement a tar-to-zstd folder compressor for many small files.”

Step 9 — Unit Tests & Benchmarks
What are we doing?
Validating correctness and measuring ratio vs throughput.
Why are we doing it?
Proves reliability; shows trade-offs for interviews.
How do we execute this?

```
# test_dfc.py
import os, pathlib, random, string
from compress import compress_file
from verify import verify, decompress

def tmp_file(text=True, n=20000):
p=pathlib.Path("tmp.txt");
if text: p.write_text("logline\n"* (n//8))
else: p.write_bytes(os.urandom(n))
return str(p)

def test_roundtrip_text():
src=tmp_file(True); man=compress_file(src, mode="auto")
assert verify(man["output"] + ".dfc.json")
decompress(man["output"])
assert pathlib.Path(src).read_bytes()==pathlib.Path(src).read_bytes()

def test_roundtrip_binary():
src=tmp_file(False); man=compress_file(src, mode="max")
assert verify(man["output"] + ".dfc.json")
```

ChatGPT regeneration prompt
“Write pytest cases that compress/verify/decompress synthetic text and random binaries.”

Step 10 — Extensions & Safety Notes
What are we doing?
Outlining real-world upgrades and guardrails.
Why are we doing it?
Prepare for large datasets, clusters, and ops hygiene.
How do we execute this?
Parallel frames: independent zstd frames per chunk with index, merged on output.

S3/GCS streaming: read/write via ranged streams; multipart uploads.

Encryption: envelope encryption (AES-GCM) layered after compression; key mgmt via KMS.

Dedup: content-defined chunking (Rabin) before compression to reduce duplicates.

Telemetry: write speed, ratio, codec decisions to CSV for later tuning.

Safety: verify before deleting sources; guard against zip-slip on decompress of archives; resource limits for untrusted inputs.

ChatGPT regeneration prompt
“List production features: parallel frames, cloud streaming, encryption, dedup, telemetry, and safety checks.”

Project Facts
Market Growth ■
Data volumes and egress costs keep rising; smart compression is a low-hanging lever in pipelines, backups, and telemetry shipping.
Cost Savings ■
Auto-choosing the right codec/level reduces storage and transfer costs; batching tiny files decreases syscall overhead.
Infrastructure Longevity ■■
A modular detector/strategy/compressor design lets you plug in future codecs (LZ4, LZSSE) or hardware offload (ISA-L) with minimal changes.
Real-World Applications ■
CI artifact packing, log archiving, data lake cold storage.

Edge gateways compressing NDJSON/CSV before upload.

Analytics exports, backup rotation windows, and developer tooling.

Industry Adoption ■
Ops teams rely on zstd for balanced speed/ratio, brotli for web assets, and xz for archival.
Auto-tuning tools bring these choices together.
Interview Question and Answer

-- 1. Interview Question: Explain your Dynamic File Compression Utility project.
-- Answer:
-- In this project, I built a file compression and decompression utility using Huffman Coding. The system reads an input file, calculates character frequencies, builds a Huffman Tree using a priority queue or min heap, generates binary codes for each character, compresses the file, and can also decompress it back to the original content. This project demonstrates important DSA concepts like trees, heaps, hash maps, greedy algorithms, and file handling.

-- 2. Interview Question: What problem does this project solve?
-- Answer:
-- This project solves the problem of reducing file size by encoding frequently occurring characters with shorter binary codes and less frequent characters with longer binary codes. This helps save storage space and improves data transfer efficiency.

-- 3. Interview Question: Which algorithm did you use for compression?
-- Answer:
-- I used Huffman Coding, which is a greedy algorithm used for lossless data compression.

-- 4. Interview Question: Why is Huffman Coding called a greedy algorithm?
-- Answer:
-- Huffman Coding is called greedy because at each step, it selects the two characters or nodes with the lowest frequency and combines them to build the tree. This local optimal choice leads to an efficient prefix code.

-- 5. Interview Question: What data structures did you use in this project?
-- Answer:

-- I used a hash map to store character frequencies, a priority queue or min heap to build the Huffman Tree, and a binary tree to generate Huffman codes.

-- 6. Interview Question: What is a Huffman Tree?
-- Answer:
-- A Huffman Tree is a binary tree where each leaf node represents a character and the path from root to leaf represents the binary code of that character.

-- 7. Interview Question: What is lossless compression?
-- Answer:
-- Lossless compression means the original data can be perfectly recovered after decompression. Huffman Coding is lossless because the decompressed file matches the original file.

-- 8. Interview Question: How do you calculate the compression ratio?
-- Answer:
-- Compression ratio can be calculated by comparing the original file size with the compressed file size. A smaller compressed file size means better compression.

-- 9. Interview Question: What challenges did you face in this project?
-- Answer:
-- The main challenges were building the Huffman Tree correctly, generating prefix-free binary codes, handling file input/output, and ensuring that decompression recreated the original file accurately.

-- 10. Interview Question: How can this project be improved further?
-- Answer:
-- This project can be improved by supporting multiple file types, adding a GUI, improving binary file handling, adding encryption with compression, showing compression statistics, and optimizing performance for large files.